```
WITH TEXT_IO; USE  TEXT_IO;¶SEPARATE(¬IDL.SEM_PHASE )¶--|---------------------------------------------------------------------------------------------
----¶--|¬¬UARITH¶--|-------------------------------------------------------------------------------------------------¶PACKAGE BODY UARITH¬IS¶    ¶  US
E UNIV_OPS, EXPRESO;¶   ¶--|-------------------------------------------------------------------------------------------------¶--|¬¬FUNCTION IS_ZERO¶FU
NCTION IS_ZERO ( V :VECTOR ) RETURN BOOLEAN IS¶BEGIN¶  RETURN V.D( 1..V.L ) = (1..V.L => 0);¶END IS_ZERO;¶   ¶--||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION U_VAL¶--|¶FUNCTION U_VAL ( A :INTEGER )¬RETURN TREE IS¶  A_SPREAD¬: VECTOR;
¶BEGIN¶  SPREAD(¬A, A_SPREAD);¶  RETURN U_INT ( A_SPREAD);¶END;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||¶--|¬¬FUNCTION U_VALUE¶--|¶FUNCTION U_VALUE ( TXT :STRING ) RETURN¬TREE IS¶BEGIN¶  RETURN EVAL_NUM( TXT );¶END;¶--||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION U_POS¶--|¶FUNCTION U_POS ( A :TREE ) RETURN INTEGER IS¶BEGIN
¶  IF (A.PT = HI AND¬A.NOTY = DN_NUM_VAL) THEN¶    IF A.NSIZ = 0 THEN¶      RETURN INTEGER( A.ABSS );¶    ELSIF¬A.NSIZ = 1 THEN¶      RETURN INTEGER(
-A.ABSS¬- 1 );¶    END IF;¶  END IF;¶  PUT_LINE( "ERR U_POS" );¶  RAISE PROGRAM_ERROR;¶END U_POS;¶--|##################################################
################################################¶--|¬¬FUNCTION U_EQUAL¶FUNCTION U_EQUAL ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VO
ID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  ELSIF (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL) OR ELSE LEFT.TY = DN_NUM_VAL THEN¶    DECLAR
E¶      L_SPREAD, R_SPREAD¬: VECTOR;¶    BEGIN¶      SPREAD( LEFT,¬L_SPREAD );¶      SPREAD( RIGHT, R_SPREAD¬);¶      IF L_SPREAD.S¬= R_SPREAD.S AND T
HEN V_EQUAL( L_SPREAD, R_SPREAD¬) THEN¶        RETURN U_VAL( 1 );¶      ELSE¶        RETURN U_VAL( 0 );¶      END¬IF;¶    END;¶  ELSE¬¬¬¬¬¬--| VALEUR R
EELLE¶    RETURN U_EQUAL(¬D( XD_NUMER, LEFT) * D( XD_DENOM, RIGHT¬), D( XD_NUMER, RIGHT ) * D( XD_DENOM, LEFT ) );¶  END IF;¶END;¶--|||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION U_NOT_EQUAL¶--|¶FUNCTION U_NOT_EQUAL ( LEFT, RIGHT :TRE
E ) RETURN¬TREE IS¶BEGIN¶  RETURN NOT U_EQUAL ( LEFT, RIGHT );¶END U_NOT_EQUAL;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||¶--|¬FUNCTION "<"¶--|¶FUNCTION "<" ( LEFT, RIGHT : TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOI
D THEN¶    RETURN TREE_VOID;¶¬  ¶  ELSIF (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL) OR ELSE LEFT.TY = DN_NUM_VAL THEN¶    DECLARE¶      L_SPREAD,
R_SPREAD¬: VECTOR;¶    BEGIN¶      SPREAD( LEFT,¬L_SPREAD );¶      SPREAD( RIGHT, R_SPREAD¬);¶      IF L_SPREAD.S¬< 0 THEN¶        IF R_SPREAD.S > 0 O
R ELSE V_LESS( R_SPREAD, L_SPREAD ) THEN¶¬RETURN U_VAL( 1 );¶        END IF;¶      ELSE¶        IF R_SPREAD.S > 0 AND¬THEN V_LESS( L_SPREAD, R_SPREAD
) THEN¶¬RETURN U_VAL( 1 );¶        END IF;¶      END¬IF;¶      RETURN U_VAL(¬0 );¶    END;¶  ELSE¬¬¬¬¬¬¬¬¬¬¬--| VALEUR REELLE¶    RETURN "<" (¬D( XD_NU
MER, LEFT )¬* D( XD_DENOM, RIGHT ),¶¬¬D( XD_NUMER, RIGHT ) * D( XD_DENOM, LEFT )¶¬     );¶  END IF;¶END "<";¶--||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "<="¶FUNCTION "<=" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  RETURN NOT (R
IGHT¬< LEFT);¶END "<=";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION ">"¶FUNCTION
 ">" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  RETURN (RIGHT < LEFT);¶END ">";¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||¶--|¬FUNCTION ">="¶FUNCTION ">=" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  RETURN NOT (LEFT < RIGHT);¶END ">=";¶--|
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION U_MEMBER¶FUNCTION U_MEMBER (¬VALUE, DIS
CRETE_RANGE :TREE )¬RETURN TREE IS¶BEGIN¶  IF VALUE = TREE_VOID THEN¶    RETURN TREE_VOID;¶  END IF;¶      ¶  IF DISCRETE_RANGE.TY = DN_RANGE THEN¶
  RETURN (VALUE >= EXPRESO.GET_STATIC_VALUE( D(¬AS_EXP1, DISCRETE_RANGE ) ) )¶¬  AND (VALUE <= GET_STATIC_VALUE( D(AS_EXP2, DISCRETE_RANGE¬) ) );¶
      ¶  ELSIF DISCRETE_RANGE.TY = DN_RANGE_ATTRIBUTE THEN¶    TEXT_IO.PUT_LINE( "!! $$$$ RANGE ATTR DISCR SUBT" );¶    RETURN TREE_VOID;¶          ¶  EL
SIF DISCRETE_RANGE.TY = DN_DISCRETE_SUBTYPE THEN¶    DECLARE¶      SUBTYPE_INDICATION¬: CONSTANT TREE¬:= D( AS_SUBTYPE_INDICATION, DISCRETE_RANGE );¶
     NAME¬¬: CONSTANT TREE¬:= D( AS_NAME, SUBTYPE_INDICATION );¶      CONSTRAINT¬¬: CONSTANT TREE¬:= D( AS_CONSTRAINT, SUBTYPE_INDICATION¬);¶    BEGIN
¶      IF CONSTRAINT.TY IN CLASS_RANGE THEN¶        RETURN U_MEMBER( VALUE, CONSTRAINT );¶      ELSIF CONSTRAINT.TY IN CLASS_REAL_CONSTRAINT AND THEN¬
D( AS_RANGE, CONSTRAINT ) /= TREE_VOID THEN¶        RETURN U_MEMBER( VALUE, D( AS_RANGE, CONSTRAINT ) );¶      ELSIF CONSTRAINT /= TREE_VOID THEN¶
    PUT_LINE( "!! $$$$ U_MEMBER: INDEX/DSCRMT CONSTRAINT" );¶        RAISE PROGRAM_ERROR;¶      END¬IF;¶¬  ¶      IF D( SM_DEFN, NAME ) =¬TREE_VOID¬TH
EN¶        RETURN TREE_VOID;¶      END¬IF;¶¬¬¬  -- (BETTER BE A DISCRETE SUBTYPE)¶      RETURN U_MEMBER( VALUE,¬D( SM_RANGE, D( SM_TYPE_SPEC,¬D( SM_DE
FN, NAME ) ) ) );¶    END;¶  END IF;¶  RETURN TREE_VOID;¶END U_MEMBER;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||¶      --|¶      FUNCTION "<="(LEFT, RIGHT: TREE)¬RETURN BOOLEAN IS¶      BEGIN¶        RETURN (LEFT <= RIGHT) = U_VAL¬( 1);¶
    END¬"<=";¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶      FUNCTION ">="(LEFT
, RIGHT: TREE)¬RETURN BOOLEAN IS¶      BEGIN¶        RETURN (LEFT >= RIGHT) = U_VAL¬( 1);¶      END¬">=";¶--|#########################################
##########################################################¶--|¶FUNCTION U_EQUAL ( LEFT, RIGHT :TREE ) RETURN BOOLEAN IS¶BEGIN¶  RETURN U_EQUAL ( LEFT,
RIGHT ) = U_VAL ( 1 );¶END U_EQUAL;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¶FUNCTION
U_MEMBER(VALUE, DISCRETE_RANGE: TREE) RETURN BOOLEAN IS¶BEGIN¶  RETURN U_MEMBER( VALUE, DISCRETE_RANGE ) = U_VAL( 1 );¶END U_MEMBER;¶--||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "AND"¶FUNCTION "AND" ( LEFT, RIGHT :TREE ) RETURN TRE
E IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  ELSIF LEFT.ABSS >¬0 AND RIGHT.ABSS > 0 THEN¶    RETURN U_VAL( 1¬);¶
  ELSE¶    RETURN U_VAL( 0¬);¶  END IF;¶END "AND";¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|¶--|¬FUNCTION "OR"¶FUNCTION "OR" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  E
LSIF LEFT.ABSS >¬0 OR RIGHT.ABSS > 0¬THEN¶    RETURN U_VAL( 1¬);¶  ELSE¶    RETURN U_VAL( 0¬);¶  END IF;¶END "OR";¶--||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION "XOR"¶FUNCTION "XOR" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF L

EFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  ELSIF LEFT.ABSS /= RIGHT.ABSS THEN¶    RETURN U_VAL( 1¬);¶  ELSE¶    RETURN U_VAL( 0
¬);¶  END IF;¶END "XOR";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION "NOT"¶FUNC
TION "NOT" ( RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  ELSE¶    RETURN U_VAL( 1¬- INTEGER( RIGHT.ABSS mod
 2 )¬);¶  END IF;¶END "NOT";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "-"¶FUN
CTION "-" ( RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶  ELSIF (RIGHT.PT =¬HI AND THEN RIGHT.NOTY = DN_NUM_V
AL) OR¬ELSE RIGHT.TY = DN_NUM_VAL THEN¶    DECLARE¶      R_SPREAD¬: VECTOR;¶    BEGIN¶      SPREAD( RIGHT, R_SPREAD¬);¶      R_SPREAD.S :=¬- R_SPREAD.
S;¶      RETURN U_INT(¬R_SPREAD );¶    END;¶  ELSE¬¬¬¬¬¬¬¬¬¬¬--| REEL¶    RETURN U_REAL (¬- D ( XD_NUMER, RIGHT ), D ( XD_DENOM, RIGHT ) );¶  END IF;¶E
ND "-";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "ABS"¶FUNCTION "ABS" ( RIGHT
 :TREE ) RETURN TREE IS¶BEGIN¶  IF RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶¬  ¶  ELSIF (RIGHT.PT =¬HI AND THEN RIGHT.NOTY = DN_NUM_VAL) OR¬ELSE R
IGHT.TY = DN_NUM_VAL THEN¶    DECLARE¶      R_SPREAD¬: VECTOR;¶    BEGIN¶      SPREAD( RIGHT, R_SPREAD¬);¶      IF R_SPREAD.S¬> 0 THEN¶        RETURN
RIGHT;¶      ELSE¶        R_SPREAD.S := +1;¶        RETURN U_INT( R_SPREAD );¶      END¬IF;¶    END;¶  ELSE¬¬¬¬¬¬¬¬¬¬¬--| DOIT ETRE UN REEL¶    RETURN
U_REAL (¬ABS D ( XD_NUMER, RIGHT ), D ( XD_DENOM, RIGHT ) );¶  END IF;¶END "ABS";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION "+"¶FUNCTION "+" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID
THEN¶    RETURN TREE_VOID;¶¬¶  ELSIF (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL) OR ELSE LEFT.TY = DN_NUM_VAL THEN¶    DECLARE¶      L_SPREAD, R_SP
READ¬: VECTOR;¶    BEGIN¶      SPREAD ( LEFT, L_SPREAD¬);¶      SPREAD ( RIGHT, R_SPREAD );¶¬      ¶      IF L_SPREAD.S¬= R_SPREAD.S THEN¶        V_ADD
 ( L_SPREAD, R_SPREAD );¶        RETURN U_INT ( R_SPREAD );¶¬      ¶      ELSIF V_EQUAL¬( L_SPREAD, R_SPREAD ) THEN¶        RETURN U_VAL ( 0 );¶¬
    ¶      ELSIF V_LESS ( L_SPREAD, R_SPREAD¬) THEN¶        V_SUB ( L_SPREAD, R_SPREAD );¶        RETURN U_INT ( R_SPREAD );¶¬      ¶      ELSE¶
    V_SUB ( R_SPREAD, L_SPREAD );¶        RETURN U_INT ( L_SPREAD);¶      END¬IF;¶    END;¶  ELSE¬¬¬¬¬¬--| REEL¶    RETURN U_REAL(¶¬      D ( XD_NUMER,
LEFT ) * D¬( XD_DENOM, RIGHT )¬+ D ( XD_NUMER, RIGHT ) * D (¬XD_DENOM,¬LEFT ),¶¬      D ( XD_DENOM, LEFT ) * D¬( XD_DENOM, RIGHT )¶¬      );¶  END IF;¶E
ND "+";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION "-"¶FUNCTION "-" ( LEFT, RI
GHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶¬  ¶  ELSIF (LEFT.PT = HI AND THEN LEFT.NOTY =
DN_NUM_VAL) OR ELSE LEFT.TY = DN_NUM_VAL THEN¶    DECLARE¶      L_SPREAD, R_SPREAD¬: VECTOR;¶    BEGIN¶      SPREAD ( LEFT, L_SPREAD¬);¶      SPREAD (
 RIGHT, R_SPREAD );¶      R_SPREAD.S :=¬- R_SPREAD.S;¶¬¬¬  -- REST¬OF CODE SAME AS +¶      IF L_SPREAD.S¬= R_SPREAD.S THEN¶        V_ADD ( L_SPREAD, R
_SPREAD );¶        RETURN U_INT ( R_SPREAD );¶¬      ¶      ELSIF V_EQUAL¬( L_SPREAD, R_SPREAD ) THEN¶        RETURN U_VAL ( 0 );¶¬      ¶      EL
SIF V_LESS ( L_SPREAD, R_SPREAD¬) THEN¶        V_SUB ( L_SPREAD, R_SPREAD );¶        RETURN U_INT ( R_SPREAD );¶      ELSE¶        V_SUB ( R_SPREAD, L
_SPREAD );¶        RETURN U_INT ( L_SPREAD );¶      END¬IF;¶    END;¶  ELSE --¬MUST BE REAL_VAL¶    RETURN U_REAL(¶¬      D ( XD_NUMER, LEFT ) * D¬( XD
_DENOM, RIGHT )¬- D ( XD_NUMER, RIGHT ) * D (¬XD_DENOM,¬LEFT ),¶¬      D ( XD_DENOM, LEFT ) * D¬( XD_DENOM, RIGHT )¶¬      );¶  END IF;¶END "-";¶--|||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION "*"¶--|¶FUNCTION "*" ( LEFT, RIGHT :TREE )
 RETURN TREE IS¶  LEFT_IS_INT¬: BOOLEAN¬:= (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL)¬OR ELSE LEFT.TY = DN_NUM_VAL;¶  RIGHT_IS_INT¬: BOOLEAN¬:= (R
IGHT.PT = HI AND THEN RIGHT.NOTY =¬DN_NUM_VAL) OR ELSE¬RIGHT.TY = DN_NUM_VAL;¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VO
ID;¶¬  ¶  ELSIF LEFT_IS_INT¬AND RIGHT_IS_INT THEN¶    DECLARE¶      L_SPREAD, R_SPREAD¬: VECTOR;¶      TEMP¬¬: VECTOR;¶    BEGIN¶      SPREAD( LEFT,¬L
_SPREAD );¶      SPREAD( RIGHT, R_SPREAD¬);¶      V_MUL( L_SPREAD, R_SPREAD, TEMP );¶      TEMP.S := L_SPREAD.S * R_SPREAD.S;¶      RETURN U_INT(¬TEMP
 );¶    END;¶  ELSIF RIGHT_IS_INT THEN¶    RETURN U_REAL( D( XD_NUMER, LEFT ) * RIGHT, D( XD_DENOM, LEFT ) );¶  ELSIF LEFT_IS_INT¬THEN¶    RETURN RIGH
T * LEFT;¶  ELSE¬¬¬¬¬¬¬¬¬¬¬--| REELS¶    RETURN U_REAL( D( XD_NUMER, LEFT ) * D( XD_NUMER, RIGHT¬), D( XD_DENOM, LEFT ) * D( XD_DENOM, RIGHT ) );¶  END
 IF;¶END "*";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "/"¶--|¶FUNCTION "/" (
 LEFT, RIGHT :TREE ) RETURN TREE IS¶  LEFT_IS_INT¬: BOOLEAN¬:= (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL)¬OR ELSE LEFT.TY = DN_NUM_VAL;¶  RIGHT_IS
_INT¬: BOOLEAN¬:= (RIGHT.PT = HI AND THEN RIGHT.NOTY =¬DN_NUM_VAL) OR ELSE¬RIGHT.TY = DN_NUM_VAL;¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THE
N¶    RETURN TREE_VOID;¶¬  ¶  ELSIF LEFT_IS_INT¬AND RIGHT_IS_INT THEN¶    DECLARE¶      L_SPREAD, R_SPREAD¬: VECTOR;¶      TEMP¬¬: VECTOR;¶    BEGIN¶
    SPREAD( RIGHT, R_SPREAD¬);¶      IF IS_ZERO( R_SPREAD ) THEN¶        RETURN TREE_VOID;¶      END¬IF;¶      SPREAD( LEFT,¬L_SPREAD );¶      V_DIV(
 R_SPREAD, L_SPREAD, TEMP );¶      TEMP.S := L_SPREAD.S * R_SPREAD.S;¶      RETURN U_INT(¬TEMP );¶    END;¶  ELSIF RIGHT_IS_INT THEN¶    IF RIGHT = U_
VAL( 0) THEN¶      RETURN TREE_VOID;¶    END IF;¶    RETURN U_REAL( D( XD_NUMER, LEFT ),¬D( XD_DENOM, LEFT )¬* RIGHT );¶  ELSE¬¬¬¬¬¬--| REEL¶    IF D(¬
XD_NUMER,¬RIGHT ) =¬U_VAL( 0 ) THEN¶      RETURN TREE_VOID;¶    END IF;¶    RETURN U_REAL( D( XD_NUMER, LEFT ) * D( XD_DENOM, RIGHT¬), D( XD_DENOM, LE
FT ) * D( XD_NUMER, RIGHT ) );¶  END IF;¶END "/";¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
¶--|¬FUNCTION "MOD"¶FUNCTION "MOD" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶¬
 ¶  ELSE¶    DECLARE¶      L_SPREAD, R_SPREAD¬: VECTOR;¶      TEMP¬¬: VECTOR;¶    BEGIN¶      SPREAD( LEFT,¬L_SPREAD );¶      SPREAD( RIGHT, R_SPREAD¬
);¶¬  ¶      IF IS_ZERO( R_SPREAD ) THEN¶        RETURN TREE_VOID;¬¬¬¬¬¬¬¬¬--| DIV ZERO¶      END¬IF;¶¬  ¶      V_DIV( R_SPREAD, L_SPREAD, TEMP );¶
  IF L_SPREAD.S¬/= R_SPREAD.S AND THEN NOT IS_ZERO( L_SPREAD ) THEN¶        V_SUB( L_SPREAD, R_SPREAD );¶        L_SPREAD.D(¬1..R_SPREAD.L ) := R_SPRE
AD.D( 1..R_SPREAD.L );¶      END¬IF;¶      L_SPREAD.S :=¬R_SPREAD.S;¶      RETURN U_INT(¬L_SPREAD );¶    END;¶  END IF;¶END "MOD";¶--||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬FUNCTION "REM"¶FUNCTION "REM" ( LEFT, RIGHT :TREE ) RETURN TREE
IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶¬  ¶  ELSE¶    DECLARE¶      L_SPREAD, R_SPREAD¬: VECTOR;¶      TEMP¬¬:

 VECTOR;¶    BEGIN¶      SPREAD( LEFT,¬L_SPREAD );¶      SPREAD( RIGHT, R_SPREAD¬);¶¬  ¶      IF IS_ZERO( R_SPREAD ) THEN¶        RETURN TREE_VOID;¬¬¬
¬¬¬¬¬¬--| DIV ZERO¶      END¬IF;¶¬  ¶      V_DIV( R_SPREAD, L_SPREAD, TEMP );¬¬¬¬¬¬¬--| LE SIGNE EST CELUI DU L_SPREAD D'ORIGINE¶      RETURN U_INT(¬L_S
PREAD );¶    END;¶  END IF;¶END "REM";¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCT
ION "**"¶FUNCTION "**" ( LEFT, RIGHT :TREE ) RETURN TREE IS¶BEGIN¶  IF LEFT¬= TREE_VOID OR RIGHT = TREE_VOID THEN¶    RETURN TREE_VOID;¶¬  ¶¬  ¶  ELSI
F (LEFT.PT = HI AND THEN LEFT.NOTY = DN_NUM_VAL) OR ELSE LEFT.TY = DN_NUM_VAL THEN¶    DECLARE¶      L_SPREAD¬: VECTOR;¶      TEMP¬: VECTOR;¶      RES
ULT¬: VECTOR;¶      COUNT¬: INTEGER¬¬:= U_POS(¬RIGHT );¶    BEGIN¶      IF COUNT < 0 THEN¶        RETURN TREE_VOID;¬¬¬¬¬¬¬¬¬--| CONSTRAINT ERROR FOR - E
XP¶      END¬IF;¶      SPREAD( LEFT,¬L_SPREAD );¶      SPREAD( 1, RESULT );¶      WHILE COUNT >¬0 LOOP¶        V_MUL( L_SPREAD, RESULT, TEMP );¶
  RESULT   :=¬TEMP;¶        RESULT.S :=¬RESULT.S * L_SPREAD.S;¶        COUNT    :=¬COUNT - 1;¶      END¬LOOP;¶      RETURN U_INT ( RESULT );¶    END;¶
¬  ¶  ELSE¶    IF U_POS( RIGHT¬) >= 0 THEN¶      RETURN U_REAL( D( XD_NUMER, LEFT ) ** RIGHT, D( XD_DENOM, LEFT ) ** RIGHT¬);¶    ELSE¶      RETURN U_
REAL( D( XD_DENOM, LEFT)¬** (- RIGHT), D( XD_NUMER, LEFT ) ** (-¬RIGHT) );¶    END IF;¶  END IF;¶END "**";¶   ¶--|------------------------------------
----------------------------------------------------------------¶END UARITH;¶
```